05

SKT K-뉴딜 아카데미 · AI· 자동화 기초 · 9/30 오전

requests api_client.py

예시 응답 대신 실제 API 응답을 받고, 요청 실패를 예외 처리합니다.

1교시 — 스스로 찾아서 공부합니다

아침 과제 문서를 열고, GitHub 웹에서 저장소와 오늘 기록까지 올립니다.

모닝 30 과제

`0930_wed/0930_morning30.md`

저장소 만들기부터 기록 두 개 확인까지

브라우저에서 할 일

security-agent-toolkit 저장소를 Public으로 만들고 README 추가

docs/ 폴더에 9/29 기록을 올리고 9/30 기록도 작성해 올리기

끝나면 docs/ 안에 기록 파일 두 개가 있는지 확인

오늘은 터미널·git 명령 없이 웹 화면만 사용합니다. 계정이 아직 없으면 먼저 가입하고, 저장소 이름은 내 컴퓨터 폴더와 맞춥니다.

IP는 인터넷에서 장치를 가리키는 주소입니다

```
# 점 세 개로 나뉜 네 덩이 숫자 · 각 칸은 0~255
203.0.113.5
```

종류	무엇	우리 로그에서는
공인 IP	인터넷에 하나뿐인 주소 — 조회하면 나라·회사가 나온다	185.220.101.34 (독일)
사설 IP	회사·집 안에서만 쓰는 주소 — 10. · 192.168. 로 시작	10.1.2.11 (내부망)

요청·응답이 어느 장치로 오갈지를 이 주소로 정합니다.
그래서 공인 IP는 조회되고, 사설 IP는 private range가 옵니다 — 다음 장에서 바로 해 봅니다.

IP 주소는 기기마다 하나씩 고정돼 있지 않습니다

주소는 네트워크 연결에 붙습니다. 공유기 뒤의 여러 기기는 같은 공인 IP 로 보일 수 있습니다.

한 기기 → 여러 IP

- 노트북 한 대가 Wi-Fi 와 유선을 같이 쓰면 주소가 둘
- 한 연결에서도 IPv4 와 IPv6 를 함께 가질 수 있다

여러 기기 → 같은 공인 IP

- 노트북·휴대폰의 사설 IP → 공유기 → 공인 IP 하나 → 인터넷
- 교실 전체가 밖에서는 같은 공인 IP 로 보인다

`ipwho.is/` 가 보는 것은 요청이 인터넷으로 나갈 때의 출발지 IP 일 뿐입니다.
IP 하나만으로 접속한 기기를 하나로 단정할 수는 없습니다.

탐지한 IP의 국가 정보를 API로 조회합니다

어제 잡은 IP 두 개가 어디 것인지 실제로 물어봅니다.

> 어제는 예시로 작성한 응답으로 연습했습니다. 오늘은 requests로 실제 API 응답을 받습니다.

```
# 어제 경보에 나온 IP
---------------------------------------------------------------------------
[추적] 185.220.101.34 → Germany (Stiftung Erneuerbare Freiheit)
[추적] 211.45.12.9 → South Korea (SamsungSDS Inc.)

# 그리고 agent_core/ 에 남는 것
---------------------------------------------------------------------------
api_client.py · api_result.json · .env.example
```

로그에 없는 IP 국가 정보는 외부 API에서 받습니다

가진 것	없는 것
어제 룰 ②가 185.220.101.34를 탐지했다	그 IP의 할당 국가와 통신사
IP 는 나라와 회사 단위로 배정된다	그걸 아는 곳은 바깥의 조회 서비스
어제는 예시 응답으로 연습했다	오늘은 실제 HTTP 요청을 보냅니다

> 어제 오후에 배운 요청·응답·상태코드를 오늘 실제로 씁니다.

HTTP 요청의 핵심 용어 세 가지를 확인합니다

용어 (한글 발음)	정의	코드 예시
requests 리퀘스츠	파이썬에서 요청을 보내는 대표 패키지	`requests.get(주소)`
params 파람스	질의 조건: API에 어떤 정보를 조회할지 지정하는 값 예: 국가 정보(country)	`params={"fields": "country"}`
headers 헤더스	HTTP 요청 헤더에 포함되는 메타데이터(추가 정보)	`headers={"Authorization": "Bearer abc123"}`

HTTP 요청 코드의 기본 작성 규칙을 확인합니다

지킬 것	왜
① 상태코드를 먼저 본다	실패한 응답의 본문은 예상한 JSON 구조와 다를 수 있다
② params로 전달한다	params에 받을 정보(contry, connection)을 넣으면 requests가 주소에 붙이고, 특수문자도 안전하게 바꿉니다
③ 인증은 headers 에	인증 정보는 HTTP 요청 헤더로 전달한다

세 규칙은 뒤의 개별 슬라이드에서 다시 설명합니다. 여기서는 전체 작성 순서를 먼저 확인합니다.

requests는 파이썬의 외부 라이브러리입니다

구분	예	쓰기 전에
파이썬에 들어 있는 것	json · re · os	import 만 하면 된다
외부 라이브러리	**requests**	설치가 먼저 · 그다음 import

VS Code 노트북:
오른쪽 위 Select Kernel에서 파이썬 환경(Python 3.x)을 선택하고
import requests를 실행합니다.

[ModuleNotFoundError가 나면]
노트북 셀을 하나 만들고 %pip install requests를 한 번 실행합니다.

선택한 커널에 requests를 설치합니다

① **Select Kernel에서 환경을 고릅니다**

노트북 오른쪽 위에서 사용할 Python 환경을 선택

② **설치 셀을 실행합니다**

```
%pip install requests
```

③ **새 셀에서 설치를 확인합니다**

`import requests` 실행 · 오류가 없으면 완료

Mac·Windows 공통 — 노트북 셀 명령은 같습니다.

https://ipwho.is/ 는 IP 정보를 JSON 으로 돌려줍니다

주소에 IP 를 안 붙이면, 지금 요청을 보낸 쪽의 IP를 조회합니다.

```
{
  "ip": "(조회된 공인 IP)",
  "success": true,
  "country": "South Korea",
  "city": "Seoul",
  "connection": {
    "isp": "SK Broadband Co Ltd"
  }
}
```

ip — 요청을 보낸 쪽의 공인 IP

success — 조회 성공 여부

country · city — IP 로 추정한 위치

connection.isp — 그 IP 의 인터넷 사업자

웹페이지처럼 꾸민 화면이 아니라 JSON 이 바로 옵니다. 이 주소가 IP 조회 API 라서, 기계가 읽기 쉬운 형식으로 돌려주는 것입니다.

`requests.get()`은 HTTP GET 요청을 보냅니다

> `requests.get`의 `get`은 **HTTP GET 메서드**입니다.
> **GET**은 서버에 저장된 정보를 조회할 때 사용합니다.

```
import requests
response = requests.get("https://ipwho.is/185.220.101.34")
print(response.status_code)
```

```
200
```

상태코드를 먼저 보고 본문은 그다음에 읽습니다

실패 응답의 본문에는 오류 정보가 들어 있습니다.
상태 확인 없이 .json() 결과의 키에 접근하면 예외가 발생할 수 있습니다.

country 값은 **Germany**입니다.
어제 여러 계정에 로그인을 시도한 IP는 독일에 할당되어 있습니다.

```
response = requests.get("https://ipwho.is/185.220.101.34")
if response.status_code == 200:
    data = response.json()
    print(data["country"])
```

```
Germany
```

`response.status_code`는 200을 반환합니다

200은 정수형 상태코드입니다.
`str(code)[0]`로 첫 자리를 확인하려면 먼저 문자열로 변환해야 합니다.

이 값(200)으로 외부 서버의 HTTP 응답을 정상적으로 수신했음을 확인합니다.

```
import requests

response = requests.get("https://ipwho.is/185.220.101.34")
print(response.status_code)
```

```
200
```

response.json()에서 country를 꺼내면 Germany가 나옵니다

.json()은 **JSON 응답 본문을 파이썬 딕셔너리로 변환합니다.**
변환한 뒤에는 딕셔너리 키로 값에 접근합니다.
response.text는 **문자열이므로 딕셔너리 키로 접근할 수 없습니다.**

```
response = requests.get("https://ipwho.is/185.220.101.34")
data = response.json()
print(data["country"])
```

```
Germany
```

requests.get()은 딕셔너리가 아니라 응답 객체를 돌려줍니다

get()이 돌려주는 것은 응답 객체입니다. .json()을 붙여야 딕셔너리가 되어 키로 값을 꺼낼 수 있습니다.

```
response = requests.get("https://ipwho.is/185.220.101.34")
print(response["country"])
```

```
TypeError: 'Response' object is not subscriptable
```

```
data = response.json()  # 딕셔너리로 바꾼다
print(data["country"])
```

```
Germany
```

response는 상태코드·헤더·본문이 다 든 상자라 키로 바로 못 꺼내고, response.json()은 본문만 딕셔너리로 바꾼 것이라 키로 꺼낼 수 있습니다.

중첩 딕셔너리의 `isp`는 `data["connection"]["isp"]`로 접근합니다

ISP(**Internet Service Provider**)는 인터넷 서비스 제공자를 뜻합니다.
여기서는 해당 IP의 통신사·네트워크 사업자입니다.
`country`는 최상위 키이고, `isp`는 `connection` 안의 중첩 키입니다.
따라서 `isp`에는 `data["connection"]["isp"]`로 접근합니다.

```
data = requests.get("https://ipwho.is/185.220.101.34").json()
print(data["country"])
```

```
Germany
```

```
print(data["connection"]["isp"])
```

```
Stiftung Erneuerbare Freiheit
```

응답에서 필요한 항목만 골라 요청합니다

IP 하나를 조회해도 여러 정보가 돌아옵니다.

IP 조회
185.220.101.34

→

API 가 줄 수 있는 것
국가 · 도시 · 좌표 · 통신사 · 우편번호 · 시간대

→

이번에 필요한 것
국가 · 통신사

경보를 해석하는 데는 국가와 통신사 둘이면 됩니다. 도시·좌표·우편번호·시간대는 이번엔 받지 않습니다.
무엇을 받을지 API 에 미리 알려 주면 그 항목만 옵니다 — 다음 장의 `params={"fields": "country,connection"}` 가 그것입니다.

쿼리 파라미터는 params로 전달합니다

- 쿼리 파라미터: 조회 결과에서 어떤 정보를 받을지 API에 덧붙여 알려주는 조건
params 가 **특수한 글자를 안전한 모양으로 바꿔 줍니다.**

쿼리 문자열을 직접 결합하면

```
ip = "185.220.101.34"
f = "country,connection"
requests.get(f"https://ipwho.is/{ip}?
fields={f}")
```

조건에 공백·한글·물음표가
들어가면 주소가 깨진다

params 로 넘기면

```
url = "https://ipwho.is/185.220.101.34"
requests.get(url,
    params={"fields":
"country,connection"}
)
```

```
{'country': 'Germany',
 'connection': {'isp': 'Hetzner Online
GmbH'}}
```

API 키는 HTTP 요청 헤더에 담습니다

```
url = "https://ipwho.is/185.220.101.34"
requests.get(url,
    headers={"Authorization": f"Bearer {token}"}
)
```

어디에	무엇을	왜
params	조회 조건	숨길 것이 아니다
headers	**API 키 · 토큰**	주소에 담으면 접속 기록에 그대로 남는다

2교시에 배운 것을 정리합니다

쓰는 법	뜻
`requests.get(주소)`	요청을 보내고 응답을 받는다
`response.status_code`	어떻게 됐는지 세 자리 숫자
`response.json()`	본문을 딕셔너리로 되돌린다
`params={...}`	조건을 안전하게 넘긴다
`headers={...}`	인증 정보를 HTTP 요청 헤더로 전달한다

⚠ **200 이어도 본문이 실패일 수 있습니다.** `success` **필드도 함께 확인하세요.**

실습 ① · 노트북 2교시 절 · 01_0930_am_requests_api_client.ipynb

문제 2-1 ~ 2-8 · 도전 ⭐2-5

① **문제 2-1 ~ 2-4 — 실제로 보내 보기**

상태코드 200 과 통신사 이름이 나오면 성공

② **문제 2-6 ~ 2-8 — params 로 필드 고르기**

필요한 응답 필드만 받아 두 IP 를 조회하면 성공

③ **인터넷을 씁니다**

안 되면 손을 드세요. 네트워크 문제와 코드 오류를 함께 확인합니다

HTTP 요청은 성공과 실패 응답을 모두 반환합니다

현실	그래서
네트워크 오류가 발생한다	서버 점검·연결 오류·요청 제한이 발생할 수 있다
실패를 예외 상황으로 보면	코드가 자꾸 멈춘다
오늘 교실에서 여러 명이 동시에 조회하면	실제로 막힐 수 있다 — 그때 쓰는 법을 지금 배운다

> 월요일에 배운 `try·except`를 외부 **API** 요청의 예외 처리에 적용합니다.

잠깐 — 수업 점검 4가지에 QR로 답해 주세요

폰 카메라로 QR을 찍으세요

① 수업 **난이도**

② **진도 속도**

③ 한 교시 **코랩 문제 수**

④ ⭐**도전 문제** 비중

> 답변은 익명입니다. 여러분이 답한 대로 남은 시간의 난이도와 속도를 맞춥니다.

외부 API 요청은 네트워크·서버·인증 문제로 실패할 수 있습니다

외부 요청에는 시간 초과·HTTP 오류·일시적 네트워크 오류가 발생할 수 있습니다.

실패 상황	프로그램 상태	처리 방법
서버가 응답하지 않는다	응답 대기가 계속된다	`timeout`
4xx·5xx 응답을 받는다	HTTP 실패 응답	`raise_for_status`
일시적 네트워크 오류가 발생한다	첫 요청이 실패한다	제한된 재시도

timeout 은 응답을 기다리는 최대 시간입니다

> timeout=5 는 5초까지만 기다린다는 뜻입니다. 그 안에 답이 안 오면 기다림을 멈추고 예외를 냅니다.

```
requests.get("https://ipwho.is/8.8.8.8", timeout=5)  # 5초까지만 기다린다
```

> timeout 이 없으면 상대가 안 죽어도 영원히 매달립니다 — 요청 한 줄에서 프로그램 전체가 멈춥니다.

끊겼을 때 잡는 법

```
try:
    r = requests.get(url, timeout=5)
except requests.Timeout:
    print("응답이 없어 건너뜁니다")
```

몇 초로 둘까

이 API 는 보통 0.3초에 답합니다.
그래서 5초면 넉넉합니다 — 정상 응답은 안 놓치고, 죽은 서버만 걸러 냅니다.
너무 짧으면 느린 정상 응답까지 끊깁니다.

HTTP 요청 실패 처리의 기본 규칙을 확인합니다

지킬 것	왜
① timeout 을 붙인다	없으면 상대가 답을 안 줄 때 영원히 기다린다
② raise_for_status() 를 부른다	이걸 불러야 4·5 로 시작하는 응답이 예외가 된다
③ 몇 번 다시 시도한다	한 번 실패가 영영 실패는 아니다. 대신 횟수에 상한을 둔다

둘째 규칙은 이전 학습과 연결됩니다 — 월요일에 배운 try·except를 외부 API 요청에 적용합니다.

timeout과 raise_for_status()로 응답 실패를 처리합니다

raise_for_status()를 호출하면 4xx·5xx 응답에서 HTTPError가 발생합니다.
발생한 예외는 except 블록에서 처리합니다.

```
url = "https://ipwho.is/185.220.101.34"
try:
    response = requests.get(url,
timeout=5
)
    response.raise_for_status()
    data = response.json()
except requests.RequestException:
    print("조회 실패")
```

status_code만 확인하면 404 응답 뒤에도 실행이 계속됩니다

> 실행은 계속됩니다.
> 404 응답을 받는 것만으로는 파이썬 예외가 발생하지 않습니다.
> print()가 404를 출력합니다.
> requests가 HTTP 응답을 정상적으로 수신했기 때문입니다.

```
response = requests.get("https://postman-echo.com/status/404", timeout=5)
print(response.status_code)
```

```
404
```

raise_for_status()는 404 응답을 HTTPError로 처리합니다

월요일에 배운 try·except를 외부 API 요청에 적용합니다.
RequestException은 requests 관련 예외의 상위 클래스입니다.
시간 초과·연결 오류·HTTP 오류를 하나의 except 블록에서 처리할 수 있습니다.

```
response = requests.get("https://postman-echo.com/status/404", timeout=5)
try:
    response.raise_for_status()
    print("성공")
except requests.RequestException:
    print("요청이 실패했습니다:", response.status_code)
```

```
요청이 실패했습니다: 404
```

여러 요청 오류를 RequestException 하나로 잡습니다

Timeout · ConnectionError · HTTPError 는 모두 RequestException 의 하위 예외입니다. 그래서 하나로 함께 잡을 수 있습니다.

Timeout
정한 시간 안에 응답 없음

ConnectionError
연결 안 됨 — DNS 실패·거부

HTTPError
raise_for_status() 뒤 4xx·5xx

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
except requests.RequestException:
    print("요청 실패")
```

⚠ 404 를 받았다고 저절로 예외가 되진 않습니다. HTTPError 로 처리하려면 raise_for_status() 를 꼭 불러야 합니다.

재시도에는 반드시 상한을 둡니다

실행 흐름을 확인합니다.
반복문은 최대 세 번 실행되며, 요청이 성공하면 값을 반환하고 함수가 종료됩니다.
요청이 실패하면 다음 반복을 실행합니다.

```
def call_with_retry(url, tries=3):
    for i in range(tries):
        try:
            r = requests.get(url, timeout=5); r.raise_for_status()
            return r.json()
        except requests.RequestException: pass
    return None
```

재시도 상한이 3이면 요청을 최대 세 번 보냅니다

range(3) 은 0·1·2 입니다.
그래서 사람이 읽을 때는 i + 1 을 붙여 1 부터 세어 줍니다.
여기에 실패했을 때만 다시 도는 조건을 붙이면 call_with_retry() 가 됩니다.
노트북 문제 3-9 이에요.

```
for i in range(3):
    print(f"{i + 1}번째 시도")
```

```
1번째 시도
2번째 시도
3번째 시도
```

API 키를 코드에 적지 않습니다

코드에 적으면

- 깃허브에 올리면 같이 올라간다
- 지워도 옛 기록에 남는다
- 사람마다 다른 키를 못 쓴다

별도 파일에 저장하면

- .env 에 적고 올리지 않는다
- 코드는 그 파일에서 읽는다
- 사람마다 자기 키를 쓴다

10/8에 깃허브로 올릴 때 `.gitignore`에 `.env`를 적습니다.
오늘은 코드와 API 키를 서로 다른 파일에 저장합니다.

.env는 저장소에서 제외하고 .env.example은 공유합니다

읽는 코드는 세 줄입니다 — 파일을 한 줄씩 읽고, =로 한 번만 나누고, 변수 이름이 일치하면 값을 가져옵니다.

.env.example은 필요한 환경변수 이름을 공유하는 예시 설정 파일입니다.

변수 이름만 있고 실제 값은 비어 있습니다.

.env — 실제 값, 저장소에서 제외

```
API_KEY=demo-key-1234
```

실제 API 키가 들어 있다

.env.example — 변수 이름, 저장소에 공유

```
API_KEY=
```

필요한 환경변수 이름만 기록

.env 파일에서 API 키 값을 읽습니다

split("=", 1) 의 1 은 「한 번만 나눠라」입니다.
값 안에 등호가 또 있어도 안 쪼개집니다.
왜 한 번만일까요? 키 값 안에 등호가 들어 있을 수 있어서입니다.
그냥 나누면 값이 잘립니다.

```
line = "API_KEY=demo-key-1234"
parts = line.strip().split("=", 1)
print(parts[1])
```

```
demo-key-1234
```

3교시 정리 ① — 실패에 대비한다

쓰는 법	뜻
`timeout=5`	5초까지만 기다린다. 반드시 붙인다
`raise_for_status()`	4xx·5xx 면 예외를 낸다. 성공하면 아무 일도 안 한다
`except requests.RequestException`	requests 가 내는 예외를 모두 잡는다
`for i in range(tries)`	몇 번 다시 시도한다

네 단계가 함께 동작합니다 — 대기 시간을 제한하고, HTTP 실패를 예외로 변환하고, 예외를 처리한 뒤, 제한된 횟수만큼 재시도합니다.

3교시에 배운 것을 정리합니다 — API 키는 코드 밖에

쓰는 법	뜻
`.env`	비밀 값을 코드 밖에 둔다
`.env.example`	값은 비우고 키 이름만 알려 준다
`split("=", 1)`	한 번만 나눈다. 키 안에 = 가 있어도 안 잘린다

> 오늘은 코드와 실제 키를 서로 다른 파일에 저장합니다.
> `.gitignore`로 `.env`를 저장소에서 제외하는 방법은 **10/8**에 배웁니다.

오후 SKT 현직자 특강 — 궁금한 것을 지금 적어 둡니다

폰 카메라로 QR을 찍으세요

✍️ 특강에서 **묻고 싶은 것**을 자유롭게 적기

💼 특히 **취업에 관한 질문**을 많이 적어 주세요

👍 다른 사람 질문 중 **나도 궁금한 것**에 좋아요

쉬는 시간에 좋아요가 많은 질문부터 추려서 특강 강사님께 전달합니다. 익명이니 편하게 적으세요.

실습 ② · 노트북 3교시 절

문제 3-1 ~ 3-9 · 도전 ⭐3-4

① **문제 3-1 ~ 3-5 — 시간 초과·HTTP 오류 처리**

없는 주소로 요청해 조회 실패가 출력되면 성공

② **문제 3-6 — `call_with_retry()`**

최대 세 번 시도한 뒤 실패 시 None을 반환하면 성공

③ **문제 3-7 ~ 3-9 — `.env` 세 개**

.env 를 만들고 읽고, .env.example 까지 만들면 성공

API 응답 딕셔너리에는 필드 21개가 있습니다

> 우리가 쓸 건 세 필드뿐입니다. 스물한 필드를 그대로 저장하면 다음 사람이 무엇을 봐야 할지 모릅니다.

```
# call_with_retry() 는 3교시에서 만든 함수입니다 (timeout · 재시도 포함)
data = call_with_retry("https://ipwho.is/211.45.12.9")
print(len(data))
```

```
21
```

필요한 API 응답 필드만 JSON 파일에 저장합니다

문제	그래서
`print()`로만 출력하면	실행 세션이 끝난 뒤 결과를 다시 사용할 수 없다
API 응답에는 필드가 21개	현재 사용할 필드는 3개
처리 방법	필요한 필드만 선택해 JSON 파일로 저장한다

> 오늘 오전의 산출물은 `api_client.py` 와 그것이 남기는 `api_result.json` 입니다.

API 응답에서 필요한 필드만 선택합니다

응답에는 필드가 스무 개가 넘습니다.
사용할 세 필드만 선택해 새 딕셔너리를 만듭니다.
결과는 어제 만든 normalized_logs.json처럼 키와 값으로 구성된 딕셔너리입니다.

```
# call_with_retry() 는 3교시에서 만든 함수입니다 (timeout · 재시도 포함)
def fetch_data(ip):
    data = call_with_retry(f"https://ipwho.is/{ip}")
    if data:
        return {"ip": ip, "country": data["country"], "isp": data["connection"]
["isp"]}
    return None
```

```
{'ip': '185.220.101.34', 'country': 'Germany', 'isp': 'Stiftung Erneuerbare
Freiheit'}
```

세 필드를 선택해 새 딕셔너리를 만듭니다

isp는 connection 안의 키이므로 대괄호를 두 번 사용합니다.
이 과정은 2교시에 배운 중첩 딕셔너리 접근입니다.
중첩 응답에서 값을 선택해 최상위 키 세 개로 구성된 새 딕셔너리를 만듭니다.

```
# call_with_retry() 는 3교시에서 만든 함수입니다 (timeout · 재시도 포함)
data = call_with_retry("https://ipwho.is/211.45.12.9")
small = {"ip": data["ip"], "country": data["country"], "isp":
data["connection"]["isp"]}
print(small)
```

```
{'ip': '211.45.12.9', 'country': 'South Korea', 'isp': 'SamsungSDS Inc.'}
```

필드 선택으로 출력 형식을 일정하게 만듭니다

그대로 21필드를 남기면	세 필드만 남기면
무엇을 봐야 할지 모른다	볼 것이 정해진다
외부 API가 필드 이름을 바꾸면 사용하는 코드 전체가 영향을 받는다	필드 선택 함수 한 곳만 수정한다
사용하지 않는 필드까지 저장된다	캡스톤 코드가 고정된 세 필드를 읽는다

> 월요일에 로그 필드 구조를 통일한 정규화와 같은 원리입니다.
> 외부 응답을 그대로 저장하지 않고 내부에서 사용할 구조로 변환합니다.

성공한 API 응답만 결과 리스트에 추가합니다

`if data:` 한 줄이 실패한 조회를 걸러 냅니다.
이걸 빼면 None 이 파일에 섞여 들어갑니다.

```
# fetch_data() 는 조회 결과에서 세 필드만 반환하는 함수입니다
results = []
for ip in ["185.220.101.34", "211.45.12.9"]:
    data = fetch_data(ip)
    if data:
        results.append(data)
print(len(results))
```

```
2
```

api_client.py가 조회 결과를 JSON 파일로 저장합니다

```
# fetch_data() 는 조회 결과에서 세 필드만 반환하는 함수입니다
# 1 · .env 에서 API 키를 읽는다 (오늘은 안 쓰지만 자리를 만들어 둔다)
# 2 · call_with_retry(url, tries=3) - timeout · raise_for_status · 재시도
# 3 · fetch_data(ip) - 필요한 세 필드만 선택해 반환한다
ips = ["185.220.101.34", "211.45.12.9"]
results = []
for ip in ips:
    row = fetch_data(ip)
    if row:
        results.append(row)
json.dump(results, f, ensure_ascii=False, indent=2)
  # api_result.json
```

POST는 티켓 내용을 서버에 보내 등록을 요청합니다

GET은 정보를 가져오고,
반대로 POST는 처리할 내용을 서버로 보냅니다.

등록할 내용
제목: 비정상 로그인 감지
우선순위: 높음

POST →

티켓 생성 API
새 티켓 생성을 요청

→

서버의 처리
이 내용으로 새 티켓을 등록

서버에 이 내용으로 새 티켓을 등록해 달라고 요청합니다.

GET · 서버의 정보를 가져옵니다　**POST** · 처리할 내용을 보내 등록·처리를 요청합니다

POST는 요청 본문으로 데이터를 전송합니다

서버에 새 티켓 생성을 요청할 때 사용합니다.
전송할 데이터는 URL이 아니라 HTTP 요청 본문에 포함합니다.
실제로는 11월 SOAR 과목에서 사용합니다.
`json=`에 딕셔너리를 전달하면 `requests`가 **JSON 직렬화와 Content-Type 헤더 설정을** 수행합니다.

```
url = "https://ipwho.is/185.220.101.34"
# 조회 - 오늘 우리가 한 것
requests.get(url, params={"fields": "country"})
# 생성 요청 - 전송할 데이터를 요청 본문에 포함한다
requests.post(url, json={"title": "의심 IP 발견", "level": "high"})
```

POST 응답에서 전송한 JSON 데이터를 확인합니다

전송한 요청 데이터를 응답 본문에 표시하는 HTTP 요청 테스트 API입니다. `json=payload`를 사용하면 `requests`가 딕셔너리를 JSON으로 직렬화하고 `Content-Type: application/json` 헤더를 설정합니다.

```
payload = {"title": "의심 IP 확인 요청", "level": "high"}
response = requests.post("https://postman-echo.com/post", json=payload, timeout=5)
print(response.json()["data"])
```

```
{'title': '의심 IP 확인 요청', 'level': 'high'}
```

cat 명령으로 저장한 JSON 파일을 확인합니다

노트북 셀 맨 앞의 !는 파이썬 코드 대신 셸 명령을 실행합니다.

indent=2로 들여쓰기했기 때문에 두 건의 JSON 데이터가 여러 줄로 출력됩니다.

```
!cat api_result.json
```

```
두 건을 포함한 여러 줄 JSON
```

api_client.py를 실행해 출력 파일을 확인합니다

.env 가 같은 폴더에 있어야 실행됩니다. 드라이브로 옮긴 뒤에도 그렇습니다.

```
# 셀을 나눠 실행합니다
!python api_client.py
```

```
[추적] 185.220.101.34 → Germany (Stiftung Erneuerbare Freiheit)
[추적] 211.45.12.9 → South Korea (SamsungSDS Inc.)
```

```
!cat api_result.json
```

```
두 건을 포함한 여러 줄 JSON
```

.env 는 올리지 않습니다

.env.example 은 필요한 키 이름만 적어 둔 예시 파일입니다. 값은 비어 있어 올려도 안전하고, 받은 사람은 이걸 보고 자기 .env 를 채웁니다.

파일	안에 무엇이	깃허브에
.env	진짜 API 키 값	올리지 않는다
.env.example	키 이름만 · 값은 비워 둔다	올린다
.gitignore	올리지 않을 파일 이름을 적어 둔 목록	올린다

오늘은 실제 값과 공유할 환경변수 이름을 서로 다른 파일에 저장합니다.
.gitignore로 .env를 저장소에서 제외하는 방법은 10/8에 배웁니다.

4교시에 배운 것을 정리합니다

쓰는 법	뜻
`fetch_data(ip)`	조회 결과에서 필요한 필드만 새 딕셔너리로 반환한다
`if data:`	실패하면 None 이라 반드시 확인한다
`json.dump(results, f, ensure_ascii=False, indent=2)`	결과를 파일로 남긴다
GET / POST	조회한다 / 새로 만든다

> 오전 산출물은 `api_client.py` · `.env.example` · `api_result.json` 셋. 오후는 최주용 강사님의 「AI 프리뷰」 입니다.

여유가 있으면 ⭐도전 문제를 풉니다

도전	무엇을 해 보나
⭐2-5 상태코드를 먼저 보기	본문을 열기 전에 200 인지 확인합니다
⭐3-4 기다리는 시간을 재 보기	`timeout=0.001` 로 일부러 실패시켜 봅니다
⭐4-8 실패한 IP 도 남기기	조회 실패한 IP 도 결과 리스트에 포함합니다

> ⭐4-8에서는 조회에 성공한 IP뿐 아니라 실패한 IP도 결과에 남겨 무엇이 조회되지 않았는지 확인합니다.

로그 수집부터 API 조회까지 데이터가 연결됩니다

줄글 로그
9/29 · 정규식

딕셔너리 16건
normalized_logs.json

경보 세 줄
9/29 · 탐지 룰

추적 두 줄
오늘 · api_result.json

모레 10/2(금)에는 웹훅과 스케줄러로 이 처리 과정을 자동 실행하는 방법을 배웁니다.

실습 ③ · 노트북 4교시 절

문제 4-1 ~ 4-8 · 도전 ⭐4-6 · ⭐4-8

① **문제 4-1 ~ 4-5 — `fetch_data()` 와 파일**

두 IP 를 모아 `api_result.json` 으로 남기면 성공

② **선택 4-7 — `api_client.py` 조립**

`agent_core` 에 파일이 저장되면 성공

③ **⭐도전 4-6 · 4-8**

추적 로그를 남기고,
조회 실패한 IP 도 결과에 포함
합니다